



Lab 4: Weather Stations Monitoring

1. Overview

The Internet of Things (IoT) is an important source of data streams in the modern digital world. The “Things” are huge in count and emit messages in very high frequency which flood the global internet. Hence, efficient stream processing is inevitable.

One use case is the distributed weather stations use case. Each “**weather station**” emits readings for the current weather status to the “**central base station**” for persistence and analysis. In this project, you will be required to implement the architecture of a weather monitoring system.

2. System Architecture

The system consists of the following stages:

2.1 Data Acquisition

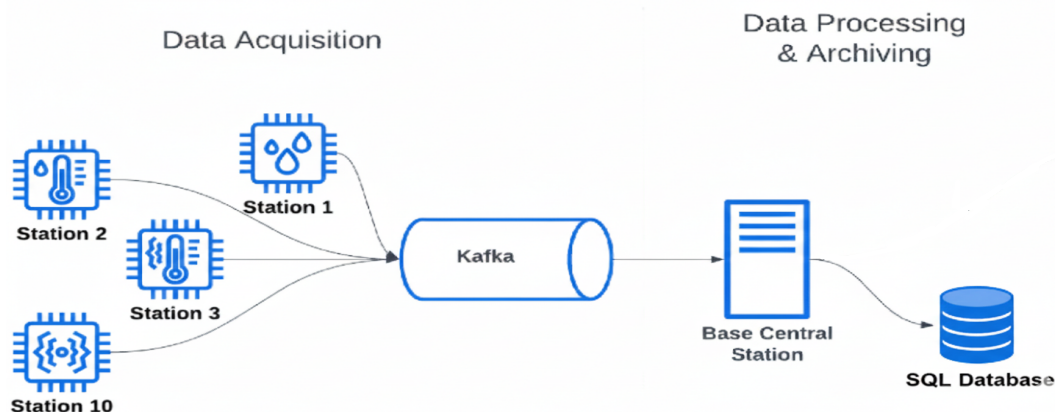
- Multiple **Weather Station Services**
- Each station periodically emits weather readings
- Readings are sent to **Apache Kafka** topics

2.2 Data Processing

- Kafka consumers and processors handle:
 - Stream ingestion
 - Filtering and transformation
 - Detection of special weather conditions (e.g., raining)

2.3 Data Persistence (SQL-based)

- All weather readings are stored in a **relational SQL database**
- Database serves as the **historical and analytical data store**
- Latest weather status per station is queried directly from the database





3. Installation

For the purpose of this lab, we will use a kubernetes application containing a cluster of:

- 10 weather stations
- 1 Kafka service + 1 Zookeeper service (for Kafka)
- 1 central base station service . To set up this cluster, you need first to:
- [Install Docker](#)
- [Pull Bitnami Kafka Docker Image](#) (Includes zookeeper)
- 1 SQL Database service (PostgreSQL / MySQL recommended)

Prerequisites:

- Docker
- Kubernetes (Minikube, Kind, or cloud-managed Kubernetes)
- Java 17+
- Kafka client libraries

4. Implementation

A) Write Weather Station Mock

Each weather station should **output a status message every 1 second** to report its sampled weather status. Check the Weather Status Message below.

To simulate data with query-able nature, you will have to:

- Randomly, change the **battery status** field by the following specs:
 - Low = 30% of messages per service
 - Medium = 40% of messages per service
 - High = 30% of messages per service
- Randomly drop messages on a 10% rate

Weather Status Message

The weather status message should be of the following schema:

```
{
  "station_id": 1, // Long
  "s_no": 1, // Long auto-incremental with each message per service
  "battery_status": "low", // String of (low, medium, high)
  "status_timestamp": 1681521224, // Long Unix timestamp
  "weather": {
    "humidity": 35, // Integer percentage
    "temperature": 100, // Integer in fahrenheit
    "wind_speed": 13, // Integer km/h
  }
}
```



B) Set up Weather Station to connect to Kafka

Use the produce API to send messages to the Kafka server. You should use the java programmatic API for this task.

To add the kafka clients dependencies, use the needed line from [this link](#).

Try getting a simple example working first before progressing. A sample produce API is as follows:

```
Properties properties = new Properties();
properties.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "127.0.0.1:9092");
properties.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,StringSerializer.class.getName());
properties.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,StringSerializer.class.getName());
try (KafkaProducer<String,String> producer = new KafkaProducer<>(properties)) {
    ProducerRecord<String, String> record = new ProducerRecord<>("my_first", "Hey Kafka!");
    producer.send(record);
}
```

You can use this to confirm this API works on your machine.

```
./bin/kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic
my_first --from-beginning
```

Output should display the message above.

C) Implement Raining Triggers in Kafka Processors

You are required to use Kafka Processors to detect when it's raining. You should use Kafka Processor to detect if humidity is **higher than 70%**. The processor should output a special message to a specific topic in Kafka.

Note : You can also use [Kafka DSL](#) to implement such features.



D) Implement Central Station

The Central Station is responsible for:

1. Consuming Kafka Streams

- Consume weather readings from Kafka
- Consume raining alerts (optional)

2. Persisting Data into SQL Database

- All received weather readings must be stored in a **relational database**
- Use **batch inserts** to avoid excessive I/O
- Recommended batch size: **5,000 records**

3. Database Schema (Minimum Required)

Example schema:

```
weather_readings (  
    id BIGSERIAL PRIMARY KEY,  
    station_id BIGINT,  
    sequence_number BIGINT,  
    battery_status VARCHAR(10),  
    timestamp BIGINT,  
    humidity INT,  
    temperature INT,  
    wind_speed INT  
)
```

E) Set up Historical Weather Statuses Analysis

You must provide SQL queries to validate:

1. Battery status distribution per station

- Confirm 30% / 40% / 30% distribution

2. Dropped messages per station

- Compare expected vs received message counts

Queries should be included in the final report.



F) Deploy using Kubernetes

You must deploy the system using Kubernetes.

Required components:

- Dockerfile for:
 - Weather Station service
 - Central Station service
- Kubernetes YAML files defining:
 - 10 Weather Station pods
 - Kafka + Zookeeper services
 - Central Station service
 - SQL Database service
 - Persistent Volumes for database storage

5. Deliverables

You are required to deliver the following:

- ☐ Full source code
- ☐ Dockerfiles
- ☐ Kubernetes YAML files
- ☐ SQL schema and analysis queries
- ☐ Final technical report (PDF)

6. Bonus

A) Cloud Deployment

You are required to deploy the system **on the cloud (without using Kubernetes)**. The objective is to demonstrate understanding of **distributed deployment, networking, and configuration in real cloud environments**.

Deployment Requirements

You must deploy the system across **at least two separate cloud virtual machines**:

1. Weather Stations Machine

- Hosts all Weather Station services
- Stations may run:
 - As multiple Docker containers, or
 - As multiple processes on the same machine
- All stations must connect remotely to Kafka

2. Central Base Station Machine

- Hosts:
 - Kafka broker and Zookeeper
 - Central Base Station service
- Consumes data from Kafka
- Writes processed data to the SQL database

Weather stations and the central base station **must not run on the same machine**.



B) Managed SQL Database Deployment

You must deploy the SQL database using a **managed database provider**.

Recommended Provider

- **Aiven Cloud**
 - PostgreSQL or MySQL
 - Free-tier or trial plan is acceptable

Requirements

- Database must be:
 - Deployed externally from application machines
 - Accessed over the internet using secure credentials
- You must:
 - Configure database connection parameters correctly
 - Avoid hardcoding secrets in source code
 - Provide proof of successful connectivity (logs or screenshots)

C) Documentation & Evidence (Mandatory for Bonus)

You must include a **Cloud Deployment Section** in your report containing:

- Cloud provider used (AWS / Azure / GCP / others)
- Machine specifications (VM size, OS)
- Network configuration (open ports, security groups)
- Deployment steps (from VM creation to running services)
- Screenshots or logs proving:
 - Stations successfully sending data
 - Central station consuming and storing data